

RC4 Implementation

```
1  /*
2  Name: Rakesh Mandal
3  Roll No: CRS2512
4  Date: 06/08/2026
5  */
6  #include <stdio.h>
7  #include <stdint.h>
8  #include <stdlib.h>
9  #include <x86intrin.h>
10
11 #define PRGA_BYTES 1048576 // 1 MB buffer for PRGA benchmarking
12
13 static inline void swap(uint8_t *a, uint8_t *b) {
14     uint8_t temp = *a;
15     *a = *b;
16     *b = temp;
17 }
18
19 // 1. Key Scheduling Algorithm (KSA)
20 void rc4_ksa(const uint8_t *key, size_t key_len, uint8_t S[256]) {
21     for (int i = 0; i < 256; i++) {
22         S[i] = (uint8_t)i;
23     }
24     uint8_t j = 0;
25     for (int i = 0; i < 256; i++) {
26         j = (j + S[i] + key[i % key_len]) & 0xFF; // modulo 256
27         swap(&S[i], &S[j]);
28     }
29 }
30
31 // 2. Pseudo-Random Generation Algorithm (PRGA)
32 void rc4_prga(uint8_t S[256], uint8_t *output, size_t len) {
33     uint8_t i = 0;
34     uint8_t j = 0;
35     for (size_t n = 0; n < len; n++) {
36         i = (i + 1) & 0xFF;
37         j = (j + S[i]) & 0xFF;
38         swap(&S[i], &S[j]);
39         output[n] = S[(S[i] + S[j]) & 0xFF];
40     }
41 }
42
43 int main(void) {
44     uint8_t S[256];
45     uint8_t key[] = "SecretKey123456789"; // 18-byte sample key
46     size_t key_len = sizeof(key) - 1;
47
48     uint8_t *prga_stream = (uint8_t *)malloc(PRGA_BYTES);
49     if (!prga_stream) {
50         perror("Allocation failed");
51         return 1;
52     }
53
54     // Measure KSA Clock Cycles
55     unsigned int aux;
56     uint64_t ksa_start = __rdtscp(&aux);
57     rc4_ksa(key, key_len, S);
58     uint64_t ksa_end = __rdtscp(&aux);
59     uint64_t ksa_cycles = ksa_end - ksa_start;
60
61     // Measure PRGA Clock Cycles
62     uint64_t prga_start = __rdtscp(&aux);
63     rc4_prga(S, prga_stream, PRGA_BYTES);
64     uint64_t prga_end = __rdtscp(&aux);
65     uint64_t prga_cycles = prga_end - prga_start;
```

```

66
67     double cycles_per_byte = (double)prga_cycles / PRGA_BYTES;
68
69     // Output Measurements
70     printf("=== RC4 Performance Measurements ===\n");
71     printf("Total Clock Cycles for KSA : %lu cycles\n", ksa_cycles);
72     printf("Total Clock Cycles for PRGA (%d bytes): %lu cycles\n", PRGA_BYTES, prga_cycles);
73     printf("Cycles Per Output Byte (PRGA): %.2f cycles/byte\n", cycles_per_byte);
74
75     free(prga_stream);
76     return 0;
77 }

```

RC4 Benchmark Results

Console Output

```

1 rakesh@rakesh-IdeaPad-3-15ALC6-Ub:~/Desktop$ gcc RC4.c
2 rakesh@rakesh-IdeaPad-3-15ALC6-Ub:~/Desktop$ ./a.out
3 === RC4 Performance Measurements ===
4 Total Clock Cycles for KSA : 11655 cycles
5 Total Clock Cycles for PRGA (1048576 bytes): 18007290 cycles
6 Cycles Per Output Byte (PRGA): 17.17 cycles/byte
7
8 rakesh@rakesh-IdeaPad-3-15ALC6-Ub:~/Desktop$ gcc -O2 RC4.c
9 rakesh@rakesh-IdeaPad-3-15ALC6-Ub:~/Desktop$ ./a.out
10 === RC4 Performance Measurements ===
11 Total Clock Cycles for KSA : 6867 cycles
12 Total Clock Cycles for PRGA (1048576 bytes): 10570539 cycles
13 Cycles Per Output Byte (PRGA): 10.08 cycles/byte
14
15 rakesh@rakesh-IdeaPad-3-15ALC6-Ub:~/Desktop$ gcc -O3 RC4.c
16 rakesh@rakesh-IdeaPad-3-15ALC6-Ub:~/Desktop$ ./a.out
17 === RC4 Performance Measurements ===
18 Total Clock Cycles for KSA : 6594 cycles
19 Total Clock Cycles for PRGA (1048576 bytes): 9399726 cycles
20 Cycles Per Output Byte (PRGA): 8.96 cycles/byte

```

Summary Table

Table 1: RC4 KSA and PRGA performance across optimization levels

Optimization	KSA (cycles)	PRGA (cycles)	Cycles/Byte
None (-O0)	11655	18007290	17.17
-O2	6867	10570539	10.08
-O3	6594	9399726	8.96

““latex ““latex

Performance Analysis

The KSA always performs a fixed 256 iterations, so its execution time is essentially constant regardless of the message length. In contrast, the PRGA executes one iteration per output byte, making *cycles per byte* (CPB) the appropriate metric for evaluating its performance.

The measured PRGA performance is **8.96 cycles/byte**. During each iteration, the algorithm updates the indices *i* and *j*, swaps the state entries *S*[*i*] and *S*[*j*], computes the output index, and generates one

keystream byte. These operations involve multiple accesses to the 256-byte state array together with simple arithmetic and masking operations.

Although the state array fits entirely in the processor’s L1 cache, each iteration depends on the results of the previous one because the state array is continuously modified. These data dependencies limit instruction-level parallelism, so compiler optimizations such as `-O3` can reduce implementation overhead but cannot remove the algorithm’s inherent sequential nature. Consequently, the measured **8.96 cycles/byte** reflects the cost of the dependent state accesses, swap operations, arithmetic, and loop overhead required to generate each keystream byte.

References

- [1] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*,
- [2] *Claude AI* (used for assistance with LaTeX formatting, benchmarking analysis, and report drafting),